IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Nantes, 13 Mars 2025
Maximiliano FLOREZ
Jules ALIANA
Amer ZAOULA
Myrella ALVES
Oussama MAKNI

Architecture logicielle pour la robotique

TP3

Encadrant: Nogueira Rafael Accacio

Sommaire

1. Introduction **3**
2. Module IMU 9 DOF Grove 101020585 **3**
3. Arduino IDE 3
3.1 Langage et bibliothèques utilisées 3
3.2 Structure d'un programme Arduino 4
4. Premier programme arduino : scanner I2C **4**
5. Récupération des accélérations **6**
5.1 Communication avec l'IMU 6
5.2 Récupération des valeurs 6
5.3 Lecture des valeurs dans le module ICM-20600 via I2C 6
6. Publisher ROSserial sur Arduino avec Raspberry Pi **8**
6.1. Lancer le serveur ROS sur la Raspberry Pi 8
6.2 Trouver le port série de l'Arduino 9
6.3 Démarrer ROSserial et connecter Arduino à ROS 10
6.4. Vérifier les messages envoyés par Arduino à ROS 11
7. Piloter la tortue avec l'IMU **11**
7.1. Communication entre l'IMU et ROS 11
7.2. Vérification de la connexion entre Arduino et ROS 12
7.3. Vérification des données IMU 13
7.4. Compilation et exécution du contrôleur de la tortue 13
7.5. Lancement de la simulation Turtlesim 14
7.6. Exécution du Node de Contrôle de la Tortue 15
7.7. Résultat final : Contrôle en temps réel 16
8. Conclusion **17**
9. annexe **17**
9.1 Annexe 1 code tortue 17
9.2 Annexe 2 code Arduino 21

1. Introduction

Dans ce TP3, nous allons explorer l'utilisation d'un module IMU 9 DOF Grove 101020585 en interaction avec une carte Arduino Mega 2560 et une Raspberry Pi exécutant ROS (Robot Operating System). L'objectif est d'établir une communication entre ces éléments via le protocole I2C et de récupérer les données de l'IMU afin de les utiliser dans un environnement ROS.

Ce projet s'articule autour de plusieurs étapes :

- La compréhension du fonctionnement du module IMU et de ses capteurs intégrés.
- L'utilisation de l'Arduino IDE pour programmer l'Arduino Mega 2560 et communiquer avec l'IMU.
- La mise en place d'un publisher ROSserial sur Arduino pour transmettre les données à la Raspberry Pi.
- L'exploitation des données du capteur IMU pour contrôler la tortue dans Turtlesim.

À la fin de ce TP, nous serons en mesure de connecter un capteur IMU à ROS, d'envoyer des données en temps réel et d'utiliser ces informations pour piloter un robot virtuel.

2. Module IMU 9 DOF Grove 101020585

Le module IMU 9 DOF Grove 101020585 est un capteur inertiel intégrant deux composants principaux :

- ICM-20600 : un gyroscope 3 axes + un accéléromètre 3 axes.
- AK09918 : un magnétomètre 3 axes.

Ces capteurs permettent d'obtenir une mesure précise du mouvement et de l'orientation d'un système robotique.

3. Arduino IDE

L'Arduino IDE est l'environnement de développement utilisé pour programmer la carte Arduino Mega 2560 et interagir avec l'IMU. Il permet d'écrire, de compiler et de téléverser du code vers l'Arduino.

3.1 Langage et bibliothèques utilisées

- Le code est écrit en C++ avec des extensions spécifiques à Arduino.
- Nous utilisons la bibliothèque Wire.h pour la communication I2C.

- Nous utilisons la bibliothèque rosserial pour envoyer les données à ROS.

3.2 Structure d'un programme Arduino

Un programme Arduino comprend deux fonctions principales :

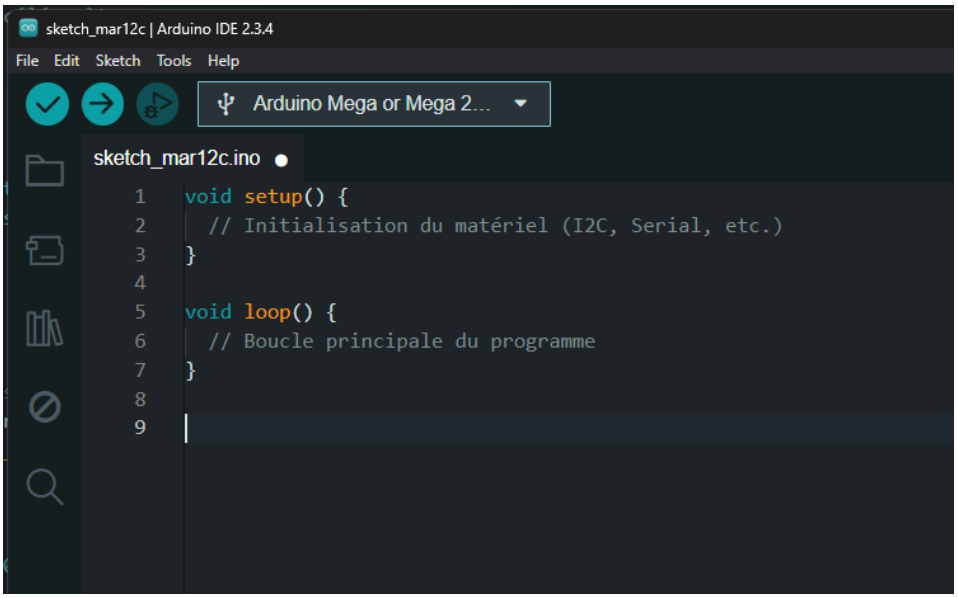

Image 1

Dans ce TP, nous allons configurer l'IMU dans `setup()` et récupérer les données du capteur dans `loop()`, pour ensuite l' envoyer à ROS.

4. Premier programme arduino : scanner I2C

Le protocole I2C est une interface bifilaire composée des signaux de données série (SDA) et d'horloge série (SCL). En général, les lignes sont à drain ouvert et bidirectionnelles. Dans une mise en œuvre généralisée de l'interface I2C, les appareils connectés peuvent être maîtres ou esclaves. Le dispositif maître place l'adresse de l'esclave sur le bus . L'ICM-20600 fonctionne toujours en tant que dispositif esclave lorsqu'il communique avec le processeur du système, qui agit donc en tant que maître. Les lignes SDA et SCL nécessitent généralement des résistances Pull up vers VDD. La vitesse maximale du bus est de 400 kHz.

L'adresse esclave de l'ICM-20600 est **b110100X**, soit 7 bits. Le bit LSB de l'adresse de 7 bits est déterminé par le niveau logique de la **broche AD0**.

```
Message (Enter to send message to 'Arduino Mega or Mega 2560' on 'COM19')

09:25:04.498 -> Scan en cours...
09:25:04.530 -> Périphérique I2C à l'adresse 0xC
09:25:04.561 -> Périphérique I2C à l'adresse 0x69
09:25:04.627 -> Fin du scan
09:25:04.627 ->
09:25:09.570 -> Scan en cours...
09:25:09.570 -> Périphérique I2C à l'adresse 0xC
09:25:09.636 -> Périphérique I2C à l'adresse 0x69
09:25:09.667 -> Fin du scan
09:25:09.667 ->
09:25:14.616 -> Scan en cours...
09:25:14.616 -> Périphérique I2C à l'adresse 0xC
09:25:14.681 -> Périphérique I2C à l'adresse 0x69
09:25:14.719 -> Fin du scan
09:25:14.719 ->
```

Image 2 : Scanner I2C

Le code commence par un appel à la méthode begin(), qui initialise la bibliothèque Wire et connecte le bus I2C en tant que contrôleur. Ensuite, il initialise le moniteur série avec un débit de 9600 baudes. Ce moniteur nous permettra d'afficher les résultats du scan des périphériques I2C connectés.

Dans la boucle loop(), on parcourt toutes les adresses I2C possibles, de 1 à 126 (puisque l'adresse 0 est réservée). Pour chaque adresse, on tente d'établir une connexion I2C avec un périphérique esclave en utilisant la méthode beginTransmission(address), puis on récupère le code d'erreur retourné par endTransmission().

Si l'erreur retournée est égale à 0, cela signifie qu'un périphérique I2C est présent à cette adresse, et son adresse hexadécimale est affichée sur le moniteur série. **(0xC = adresse de AK09918 et 0x69 = adresse ICM-20600 gyroscope 3 axes + accéléromètre)**
Si l'erreur retournée est égale à 4, cela indique une erreur non précisée lors de la transmission, et un message d'erreur est affiché pour cette adresse.

À la fin de la boucle, si aucun périphérique I2C n'a été détecté, un message indiquant l'absence de périphériques est affiché. Enfin, un délai de 5 secondes (delay(5000)) est inséré avant de recommencer le scan.

5. Récupération des accélérations

Dans cette partie, nous sommes censés créer un code permettant de récupérer les 3 accélérations que peut fournir le module IMU 9 DOF. Afin d'établir une communication entre le module et l'Arduino Mega 2560, on va utiliser la librairie Wire et les registres .

5.1 Communication avec l'IMU

Ce module utilise l'interface I2C pour échanger des données avec le microcontrôleur. L'IMU est configuré comme un périphérique esclave avec l'adresse I2C 0x69 (ou 0x68 selon le câblage de la broche AD0).

5.2 Récupération des valeurs

- Accéléromètre : fournit les accélérations en X, Y et Z (en g).
- Gyroscope : fournit la vitesse angulaire en X, Y et Z (en °/s).
- Magnétomètre : fournit l'orientation par rapport au champ magnétique terrestre.

Nous allons exploiter ces données pour contrôler un robot sous ROS.

5.3 Lecture des valeurs dans le module ICM-20600 via I2C

La communication avec l'IMU ICM-20600 suit le protocole I2C où l'IMU fonctionne en tant que périphérique esclave et le microcontrôleur en tant que maître.

Étapes du processus de lecture :

1. Condition de départ **(S - START)**

- Le maître génère un signal START (S), indiquant le début de la communication sur le bus I2C.

2. Envoi de l'adresse du périphérique avec bit d'écriture **(AD+W)**

- Le maître envoie l'adresse I2C de l'ICM-20600 (b110100X selon l'état de la broche AD0), suivie d'un bit d'écriture (W = 0).
- Le module IMU répond avec un ACK (acquittement), confirmant la réception de l'adresse.

3. Envoi de l'adresse du registre à lire (RA - Register Address)

- Le maître envoie l'adresse du registre contenant la donnée souhaitée (RA).
- L'IMU accusé de réception avec un ACK.

4. Condition de redémarrage (S - Repeated START)

- Pour passer en mode lecture, le maître génère un nouveau signal START (Sr) sans envoyer un signal STOP.

5. Envoi de l'adresse du périphérique avec bit de lecture (AD+R)

- Le maître envoie de nouveau l'adresse du périphérique, cette fois avec le bit de lecture (R = 1).
- L'IMU répond par un ACK et prépare la donnée demandée.

6. Réception de la donnée (DATA)

- L'IMU transmet le contenu du registre au maître (DATA).
- Le maître signale la fin de la transmission en envoyant un NACK (non-acquittement).

7. Condition d'arrêt (P - STOP)

- Le maître envoie un signal STOP (P), libérant ainsi le bus I2C pour d'autres communications.

Ce protocole assure une récupération fiable des valeurs stockées dans les registres internes du capteur, permettant de lire les valeurs d'accélération et de vitesse angulaire du module IMU.

```
Accélération X: 0.00 g, Y: -0.04 g, Z: 1.00 g
Accélération X: 0.00 g, Y: -0.04 g, Z: 1.00 g
Accélération X: 0.00 g, Y: -0.04 g, Z: 1.00 g
Accélération X: 0.01 g, Y: -0.04 g, Z: 1.00 g
Accélération X: 0.00 g, Y: -0.04 g, Z: 1.00 g
Accélération X: 0.01 g, Y: -0.04 g, Z: 1.00 g
Accélération X: 0.00 g, Y: -0.04 g, Z: 1.00 g
Accélération X: 0.00 g, Y: -0.04 g, Z: 1.00 g
Accélération X: 0.00 g, Y: -0.03 g, Z: 1.00 g
Accélération X: 0.01 g, Y: -0.04 g, Z: 1.00 g
Accélération X: 0.00 g, Y: -0.04 g, Z: 1.00 g
Accélération X: 0.00 g, Y: -0.04 g, Z: 1.00 g
Accélération X: 0.00 g, Y: -0.04 g, Z: 1.00 g
Accélération X: 0.01 g, Y: -0.04 g, Z: 1.00 g
Accélération X: 0.00 g, Y: -0.04 g, Z: 1.00 g
Accélération X: 0.01 g, Y: -0.04 g, Z: 1.00 g
Accélération X: 0.01 g, Y: -0.04 g, Z: 1.00 g
Accélération X: 0.01 g, Y: -0.04 g, Z: 1.00 g
```

Image 3

6. Publisher ROSserial sur Arduino avec Raspberry Pi

Dans cette partie, l'objectif est de déporter un publisher ROS sur une carte Arduino Mega 2560 et d'envoyer des messages via ROSserial à une Raspberry Pi exécutant ROS. Les captures d'écran obtenues via PuTTY montrent le processus complet.

Les images montrent que tu as bien suivi les étapes nécessaires pour connecter Arduino à ROS via ROSserial. Voici une analyse détaillée :

6.1. Lancer le serveur ROS sur la Raspberry Pi

La commande exécutée :

```
roscore
```

ROS a été démarré avec succès sur la Raspberry Pi, mettant en place le serveur ROS via **roslaunch**.

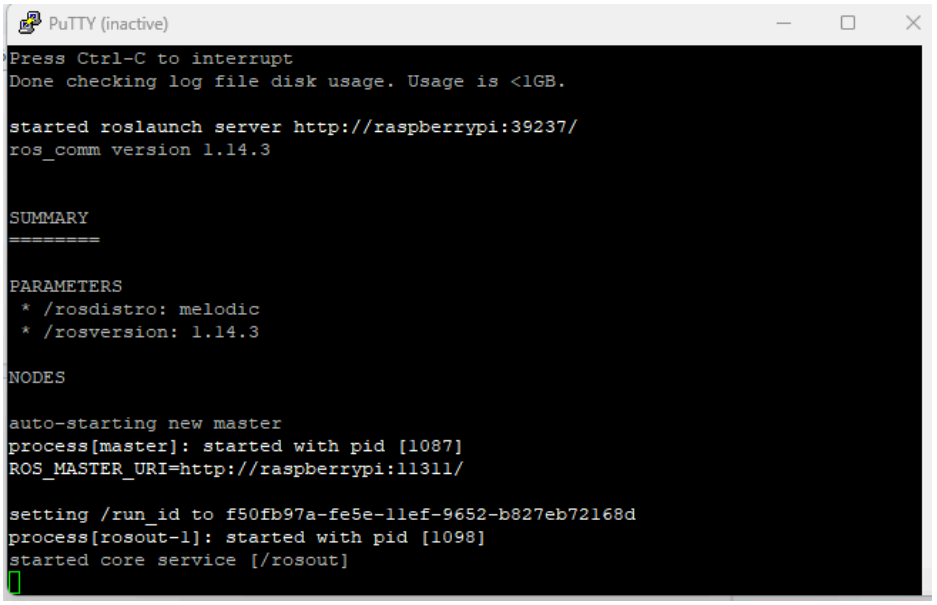

Image 4

6.2 Trouver le port série de l’Arduino

La commande utilisée :

ls /dev/tty*

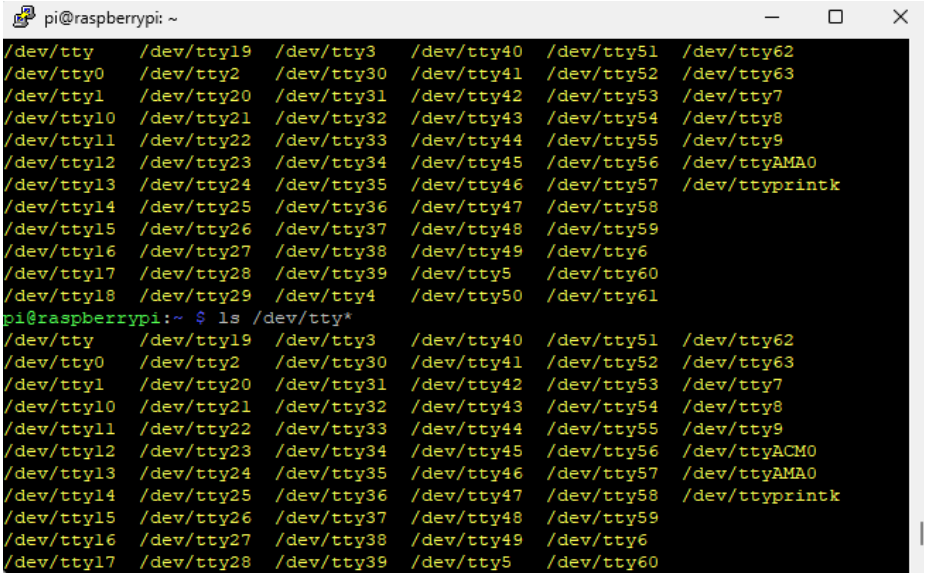

Image 5

Tous les ports séries détectés sont affichés, notamment :

- **/dev/ttyACM0** (Port USB utilisé pour Arduino)
- **/dev/ttyAMA0** (Port UART natif de la Raspberry Pi)

L'Arduino est bien détecté sous **/dev/ttyACM0**, c'est ce port qu'on va utiliser pour connecter **ROSserial**.

6.3 Démarrer ROSserial et connecter Arduino à ROS

La commande exécutée :

rosrun rosserial_python serial_node.py /dev/ttyACM0

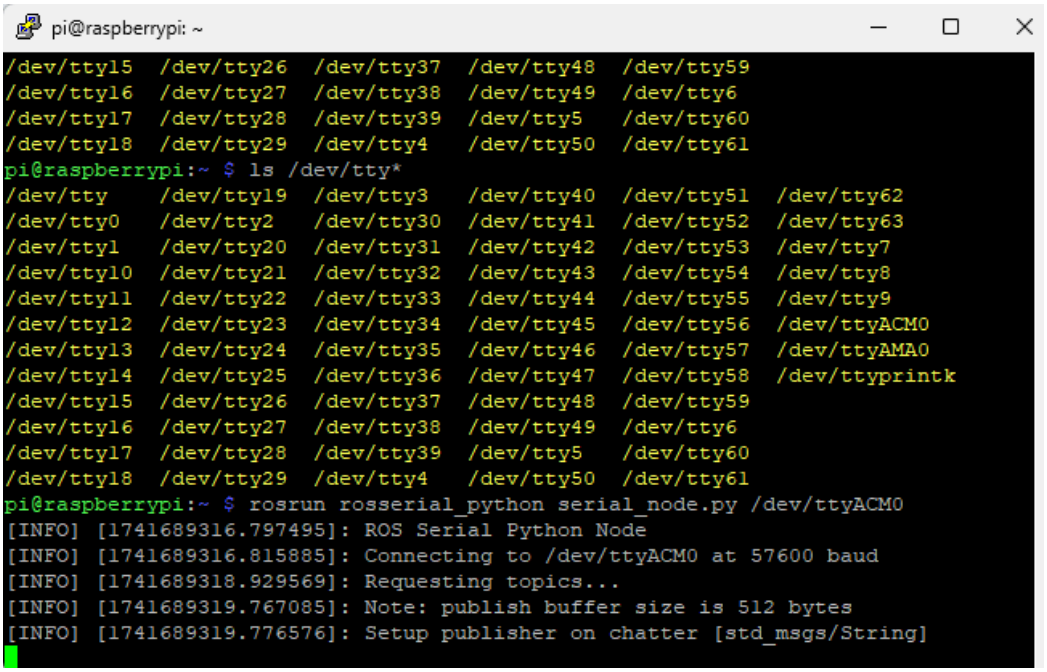

Image 6

La sortie affiche :

[INFO]: Connecting to /dev/ttyACM0 at 57600 baud
[INFO]: Requesting topics…
[INFO]: Setup publisher on chatter [std_msgs/String]

Cela signifie que :

- La connexion avec **Arduino via** `/dev/ttyACM0` est bien établie.
- Le nœud ROSserial a **créé un publisher sur le topic** `/chatter`.

6.4. Vérifier les messages envoyés par Arduino à ROS

La commande exécutée :

rostopic echo /chatter

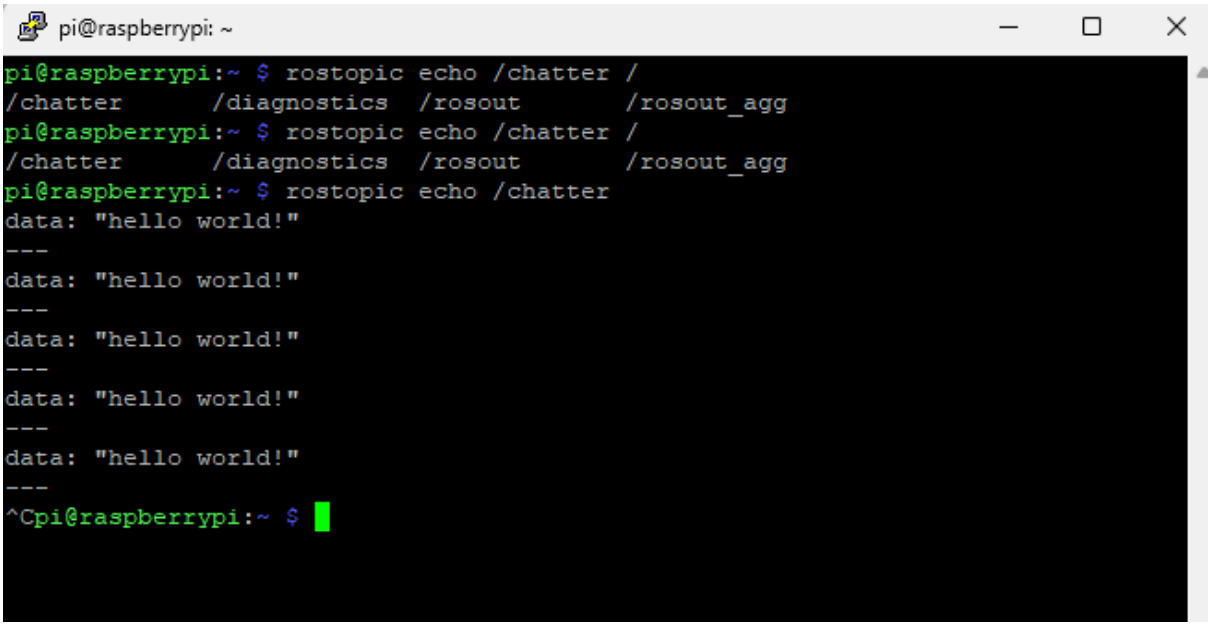

Image 7

7. Piloter la tortue avec l'IMU

Dans cette dernière partie du TP, nous utilisons les données issues du capteur IMU 9 DOF Grove 101020585 pour piloter la tortue sous Turtlesim. Grâce à la connexion entre Arduino, ROS et Turtlesim, nous sommes en mesure de transformer les mouvements du capteur en commandes de vitesse et d'orientation pour la tortue.

7.1. Communication entre l'IMU et ROS

L'IMU est connecté à l'Arduino Mega 2560 via le protocole I2C et envoie des données d'accélération (X, Y, Z) via ROSserial.

Les étapes principales de la communication sont :

1. **Arduino publie les valeurs de l'IMU** sous forme d'un message geometry_msgs/Vector3 sur le topic **/imu_accel**.
2. **ROSserial sur Raspberry Pi** reçoit ces données et les intègre dans l'architecture ROS.
3. **Un node ROS interprète ces valeurs** et les convertit en commandes de mouvement pour la tortue.
4. **La tortue bouge en fonction de l'IMU**, permettant un contrôle en temps réel.

7.2. Vérification de la connexion entre Arduino et ROS

Dans la première étape, nous devons nous assurer que ROSserial fonctionne correctement et que le node de publication des données IMU est bien lancé.

Commande pour lancer ROSserial :

```
rosrun rosserial_python serial_node.py /dev/ttyACM0
```

Comme on le voit dans l'image suivante, le publisher sur /imu_accel est bien créé et attend de publier les données de l'IMU.

Résultat attendu :

[INFO]: ROS Serial Python Node
[INFO]: Connecting to /dev/ttyACM0 at 57600 baud
[INFO]: Setup publisher on imu_accel [geometry_msgs/Vector3]

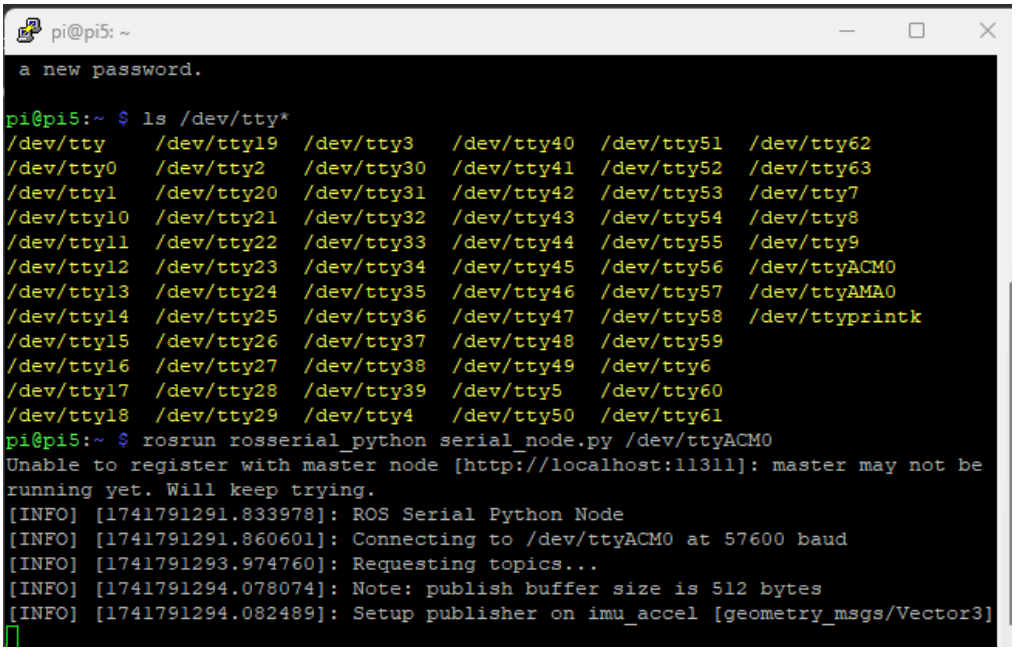

Image 8 : Confirmation de la connexion avec `/dev/ttyACM0`

7.3. Vérification des données IMU

Une fois la connexion établie, nous pouvons vérifier que les données de l'IMU sont bien publiées en exécutant la commande :

```
rostopic echo /imu_accel
```

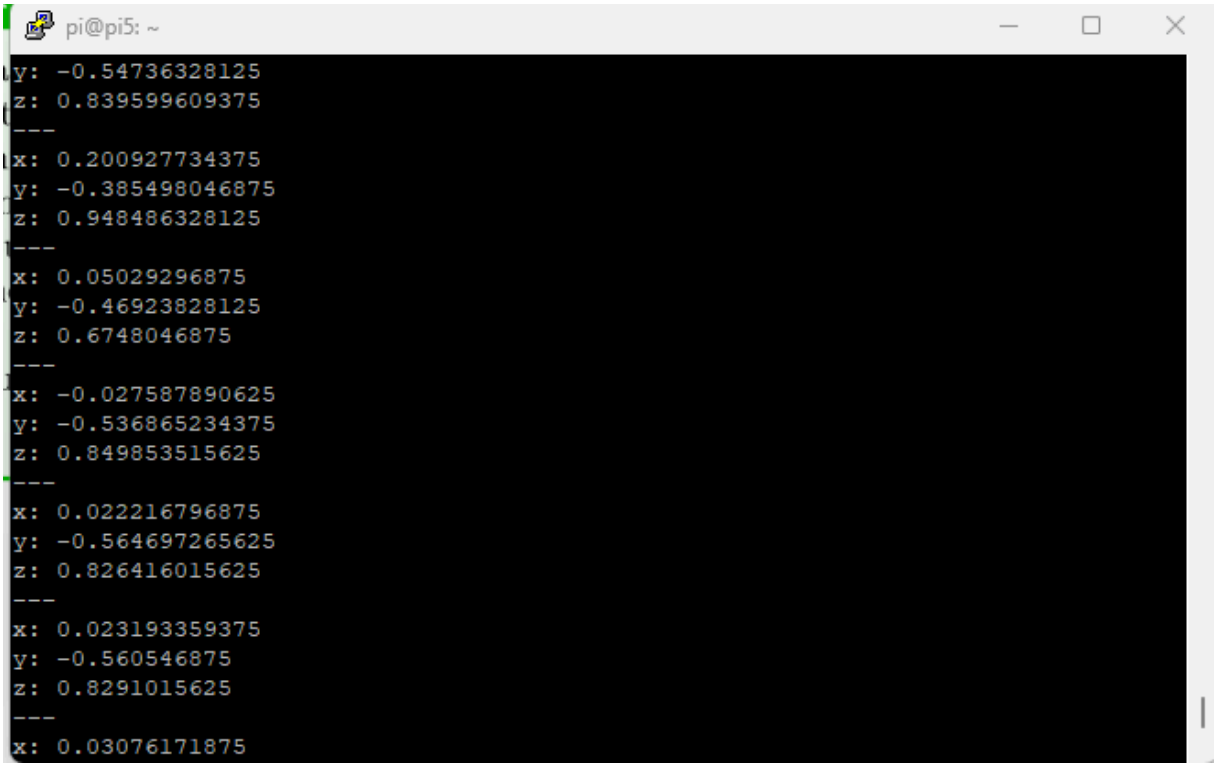

Image 9 **Résultat attendu (extrait du terminal)**

Ces valeurs représentent les accélérations mesurées en g (gravité terrestre) sur les axes X, Y et Z.

7.4. Compilation et exécution du contrôleur de la tortue

Une fois les données IMU disponibles dans ROS, nous devons les intégrer dans le programme de contrôle de la tortue.

Nous avons préparé un package contenant le fichier ControlTurtle.cpp. Après avoir modifié le code, nous devons compiler le package avec :

```
catkin_make
```

Résultat attendu après compilation réussie :

[100%] Linking CXX executable /home/pi/my_ws/devel/lib/tp2test/ControlTurtle

[100%] Built target ControlTurtle

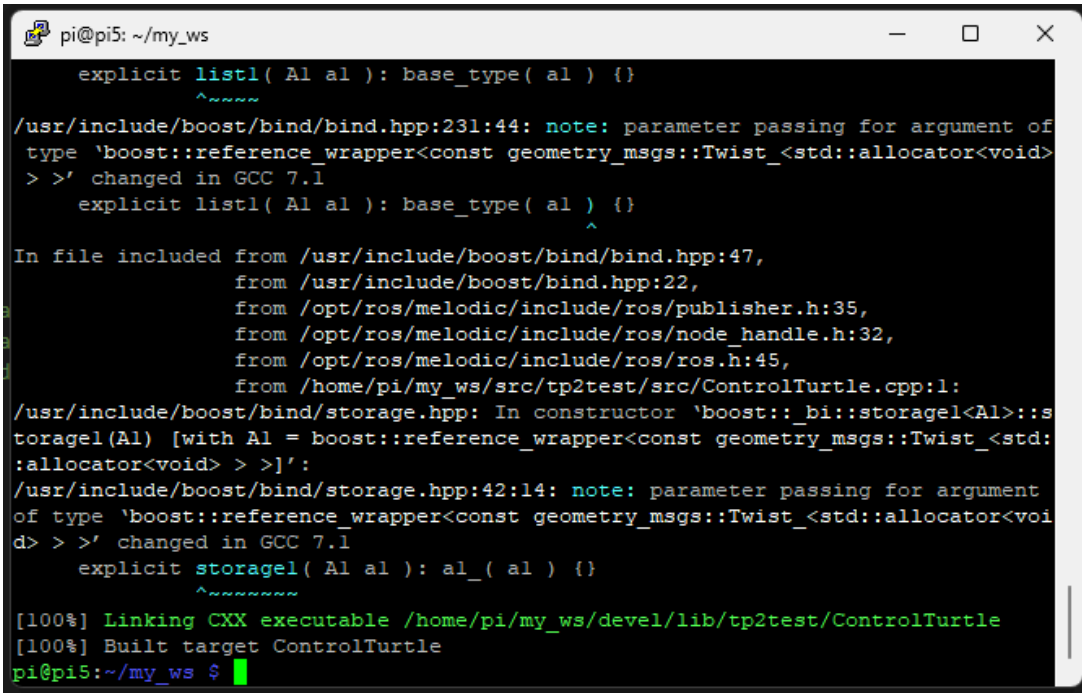

Image 9 Compilation réussie du node `ControlTurtle`

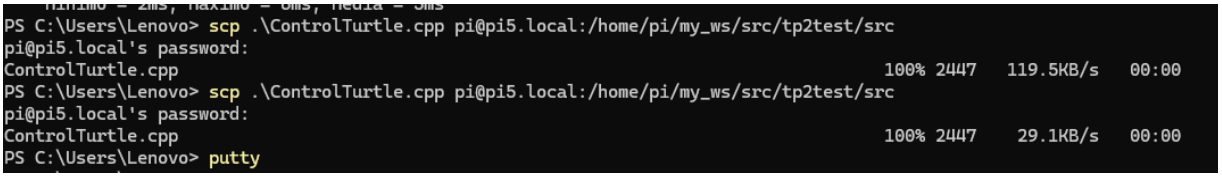

Image 10 Mise à jour du code de ControlTurtle

7.5. Lancement de la simulation Turtlesim

Maintenant que tout est prêt, nous devons lancer le simulateur Turtlesim et exécuter notre node de contrôle basé sur l'IMU.

Lancer Turtlesim :

rosrun turtlesim turtlesim_node __name:=myturtle

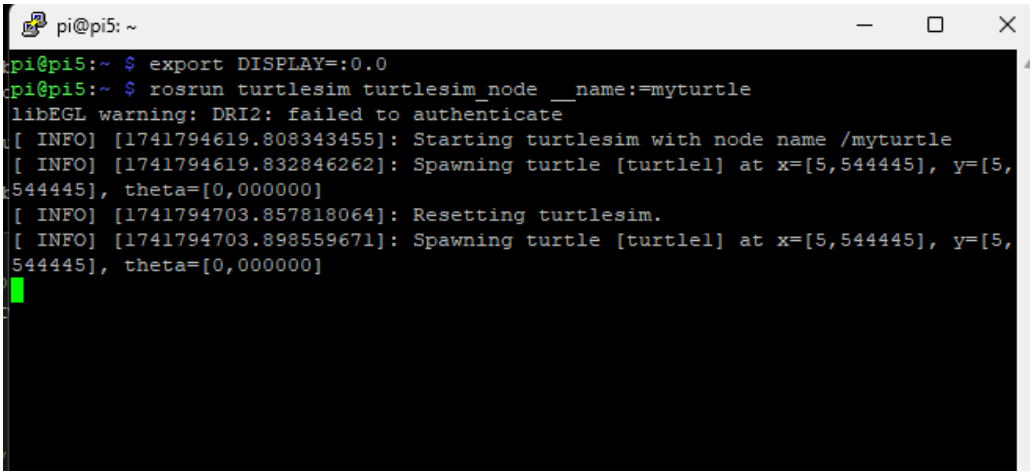

Image 11 Lancement de Turtlesim et réinitialisation de la tortue

7.6. Exécution du Node de Contrôle de la Tortue

Le node ControlTurtle écoute les données IMU et envoie les commandes de vitesse à Turtlesim.

rosrun tp2test ControlTurtle

Ce que fait ce node :

1. Convertit les valeurs IMU en vitesse linéaire et angulaire.
2. Publie ces vitesses sur /turtle1/cmd_vel.
3. La tortue se déplace en fonction de l'inclinaison du capteur.

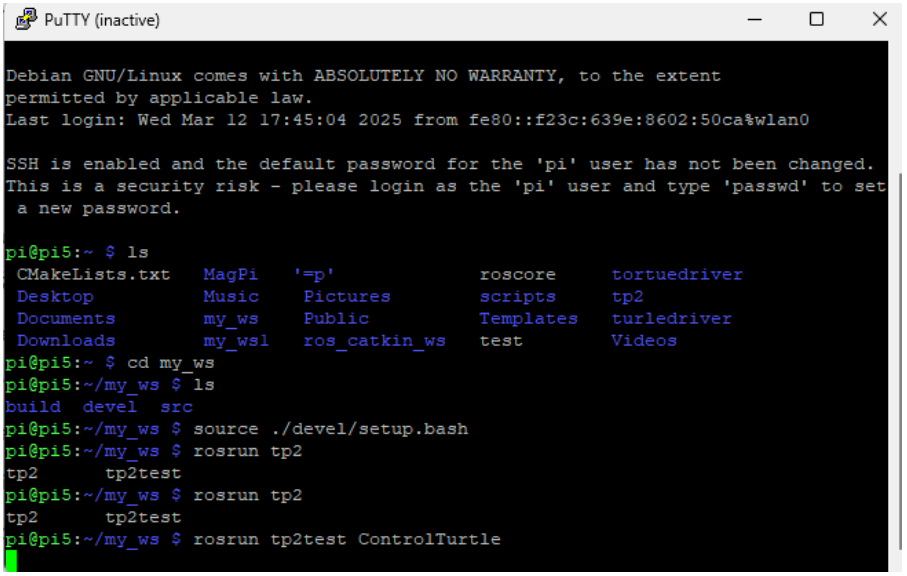

Image 12

7.7. Résultat final : Contrôle en temps réel

Ce que l'on observe à la fin :

- Si l'IMU est incliné vers l'avant, la tortue avance.
- Si l'IMU est incliné vers la gauche/droite, la tortue tourne.
- Les mouvements sont fluides et contrôlés en temps réel

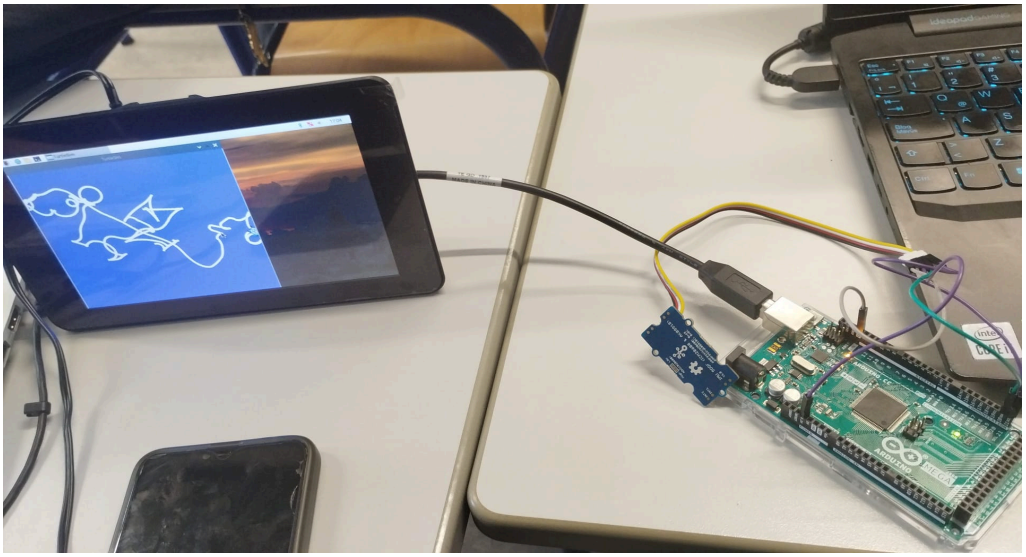

Image 13

8. Conclusion

Ce TP nous à permis de contrôler la tortue de la turtlesim en utilisant le module **IMU 9 DOF Grove 101020585** avec **un Arduino Mega 2560**. Les principales étapes à suivre étaient :

- Lire et comprendre les datasheets des capteurs ICM-20600 (gyroscope + accéléromètre) et AK09918 (boussole).
- Utiliser la bibliothèque Arduino Wire pour établir la communication I2C et récupérer les mesures.
- Créer une node ROS envoyer sur l'Arduino via `rosserial`.
- Transmettre les données IMU vers un Raspberry Pi exécutant ROS.
- Contrôle de la Tortue avec l'IMU.

Le travail effectué a reposé sur la maîtrise du protocole I2C, la capacité à comprendre la documentation et la production d'un code clair et structuré.

9. annexe

9.1 Annexe 1 code tortue

```
#include <ros/ros.h>

#include <turtlesim/Pose.h>

#include <turtlesim/SetPen.h>

#include <geometry_msgs/Twist.h>

#include <geometry_msgs/Vector3.h>

#include <std_srvs/Empty.h>

#include <math.h>

#include <stdlib.h>

#include <iostream>
```

```
// Constantes

const float pi = 3.14159265;

const float SCALE_LINEARX = 1.0;   // Facteur d'échelle pour la vitesse
linéaire en X

const float SCALE_LINEARY = 1.0;   // Facteur d'échelle pour la vitesse
linéaire en Y

const float SCALE_ANGULAR = 2.0;   // Facteur d'échelle pour la vitesse
angulaire



// Variables globales

geometry_msgs::Vector3 imu_accel;  // Stocke l'accélération reçue du capteur
IMU

ros::Publisher twist_pub;          // Déclare un éditeur ROS pour publier
des commandes de mouvement



// Callback pour recevoir les données de l'IMU envoyées par Arduino

void imuCallback(const geometry_msgs::Vector3::ConstPtr& msg) {

    imu_accel = *msg;  // Mise à jour de la variable globale avec les
nouvelles données de l'IMU



    // Conversion de l'accélération en vitesse pour la tortue

    geometry_msgs::Twist twist_msg;

    twist_msg.linear.x = imu_accel.x * SCALE_LINEARX;   // Déplacement
linéaire basé sur l'accélération en X

    twist_msg.linear.y = imu_accel.y * SCALE_LINEARY;   // Déplacement
linéaire basé sur l'accélération en Y

    twist_msg.angular.z = imu_accel.z * SCALE_ANGULAR;  // Rotation basée
sur l'accélération en Z
```

```
    // Publication de la commande de mouvement
    twist_pub.publish(twist_msg);
}

////////////////////////////////
////////////////////////////////

int main(int argc, char** argv) {
    // Initialisation du nœud ROS
    ros::init(argc, argv, "imu_turtle_control");
    ros::NodeHandle nh;

    // Abonnement au topic "imu_accel" pour recevoir les données de l'IMU
    ros::Subscriber imu_sub = nh.subscribe("imu_accel", 10, imuCallback);

    // Création du publisher pour envoyer des commandes de mouvement à la
tortue
    twist_pub = nh.advertise<geometry_msgs::Twist>("turtle1/cmd_vel", 10);

    // Client pour réinitialiser la simulation
    ros::ServiceClient reset_client =
nh.serviceClient<std_srvs::Empty>("reset");
    std_srvs::Empty reset_srv;
    reset_client.waitForExistence();  // Attente de la disponibilité du
service
    reset_client.call(reset_srv);     // Appel du service de
réinitialisation
```

```
    // Client pour modifier la couleur du crayon de la tortue

    ros::ServiceClient pen_client =
nh.serviceClient<turtlesim::SetPen>("turtle1/set_pen");

    pen_client.waitForExistence();  // Attente de la disponibilité du
service


    // Configuration du crayon (couleur et épaisseur)

    turtlesim::SetPen pen;

    pen.request.r = 255;  // Rouge

    pen.request.g = 255;  // Vert

    pen.request.b = 255;  // Bleu (Blanc en combinant les trois)

    pen.request.width = 5;  // Épaisseur du crayon

    pen.request.off = 0;  // Activer le crayon

    pen_client.call(pen);  // Envoi de la configuration au service


    // Maintien du nœud en exécution pour traiter les messages entrants

    ros::spin();


    return 0;

}
```

9.2 Annexe 2 code Arduino

```
// Adresse I2C du module IMU (vérifié avec scanner)

#define IMU_ADDRESS 0x69

// Registres de l'accéléromètre

#define ACCEL_XOUT_H 0x3B

#define ACCEL_YOUT_H 0x3D

#define ACCEL_ZOUT_H 0x3F


#include <ros.h>

#include <std_msgs/String.h>

#include <geometry_msgs/Vector3.h>

#include <Wire.h>




ros::NodeHandle nh;


// Déclaration du message ROS pour envoyer les accélérations

geometry_msgs::Vector3 accel_msg;

ros::Publisher accel_pub("imu_accel", &accel_msg);


void setup() {

    Wire.begin();

    Serial.begin(9600);

    nh.initNode();

    nh.advertise(accel_pub);
```

```
    initIMU();
}

void loop() {
    float ax, ay, az;
      ax = getAcceleration(ACCEL_XOUT_H);
      ay = getAcceleration(ACCEL_YOUT_H);
      az = getAcceleration(ACCEL_ZOUT_H);
      accel_msg.x = ax ;
      accel_msg.y = ay ;
      accel_msg.z = az ;
    // Publication des données sur le topic ROS
    accel_pub.publish(&accel_msg);

    // ROS Spin
    nh.spinOnce();

    // Petite pause pour éviter une surcharge du bus
    delay(100);
}

// Initialisation du capteur IMU
void initIMU() {
    Wire.beginTransmission(IMU_ADDRESS);
    Wire.write(0x6B);  // Registre d'alimentation (Power Management 1)
```

```
    Wire.write(0x00);  // Réveil du capteur

    Wire.endTransmission();

}


// Lire une valeur d'accélération et la convertir en "g"

float getAcceleration(uint8_t reg) {

    int16_t rawData = readIMU(reg);

    return rawData / 16384.0;  // Conversion pour ±2g

}


// Fonction de lecture d'un registre 16 bits

int16_t readIMU(uint8_t reg) {

    Wire.beginTransmission(IMU_ADDRESS);

    Wire.write(reg);

    Wire.endTransmission(false);

    Wire.requestFrom(IMU_ADDRESS, 2);

    return (Wire.read() << 8) | Wire.read();  // MSB + LSB

}
```